TechBash 2026

A Developer's Guide to Open Source Web Mapping Libraries

Build Your Own City Explorer
Leaflet, Esri Leaflet, GeoJSON, and a separate MapLibre lab

map data UI state

Courtney Yatteau, Esri





Open your preferred code editor.
We'll get started shortly!

WHERE WE ARE GOING

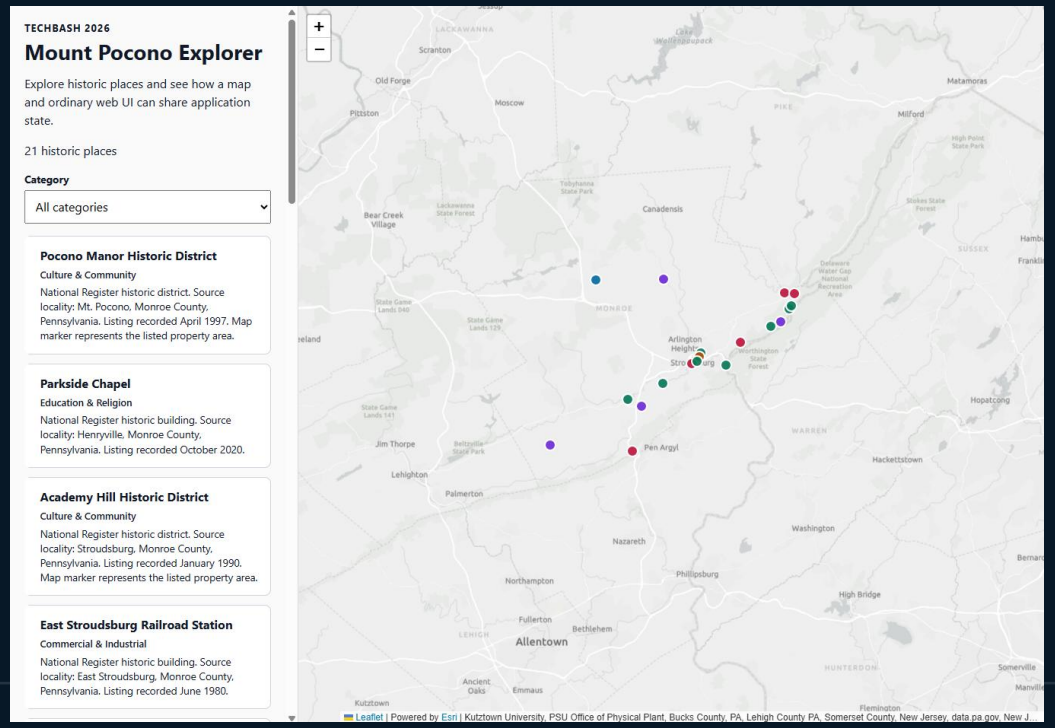
Two apps. One dataset.

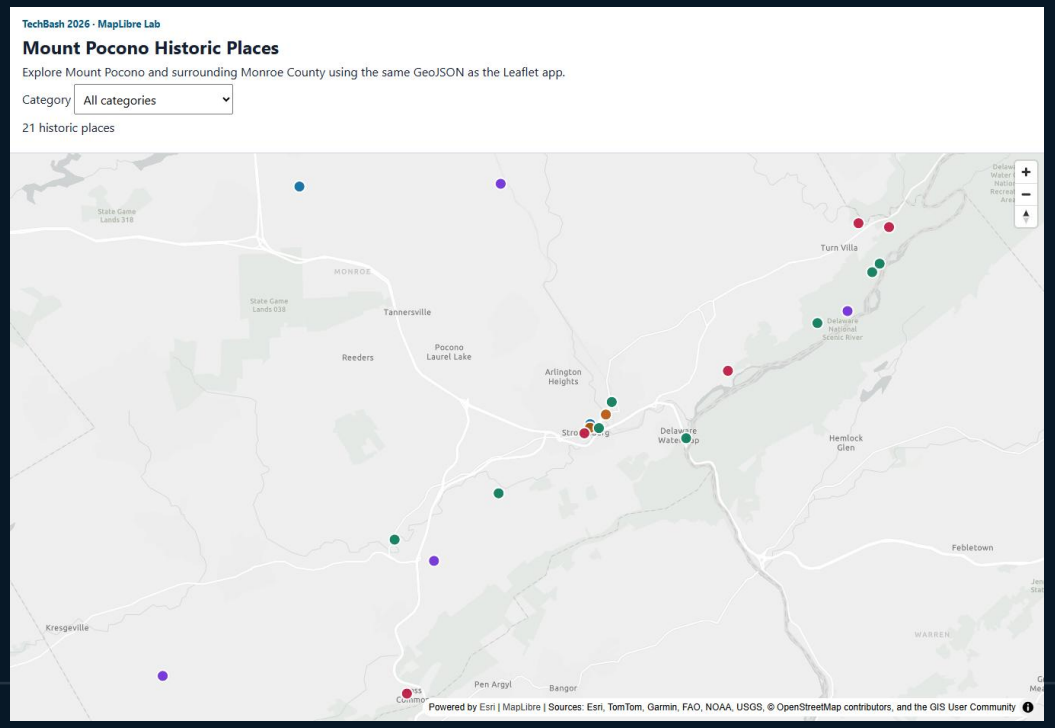
21 historic places around Mount Pocono and Monroe County

Shared GeoJSON

Leaflet City Explorer

MapLibre Lab





WORKSHOP PLAN

Our Workshop Roadmap

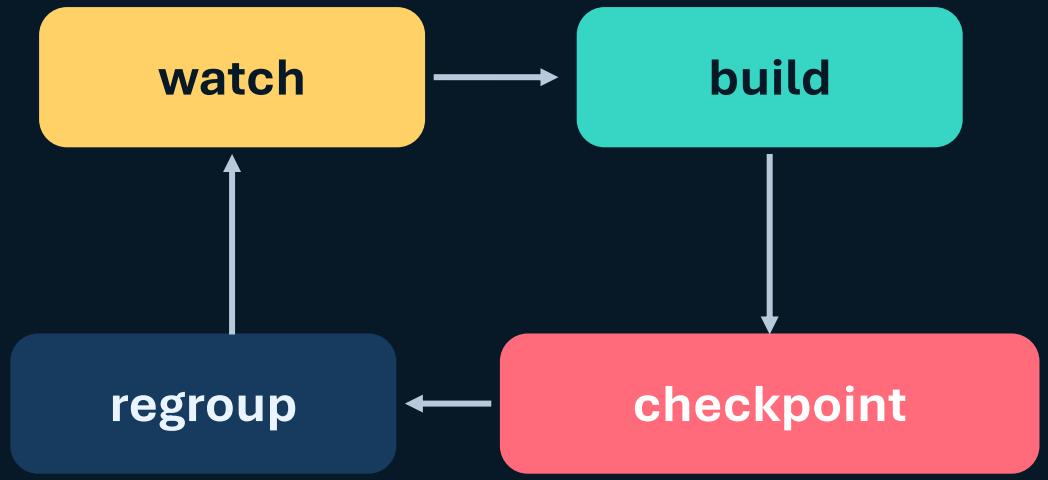
210 minutes, including explanations, builds, and checkpoints

Part	Focus
01	Setup + mapping foundations
02	Build our first map
03	Make it interactive
04	Polish the experience
05	Explore MapLibre
06	Wrap-up and extensions

WORKFLOW

How today works

I explain the idea. You follow the workshop site.
We stop at checkpoints so nobody gets stranded.



START-HERE

Open Start Here and get these three things ready.

working folder
config.js
local server

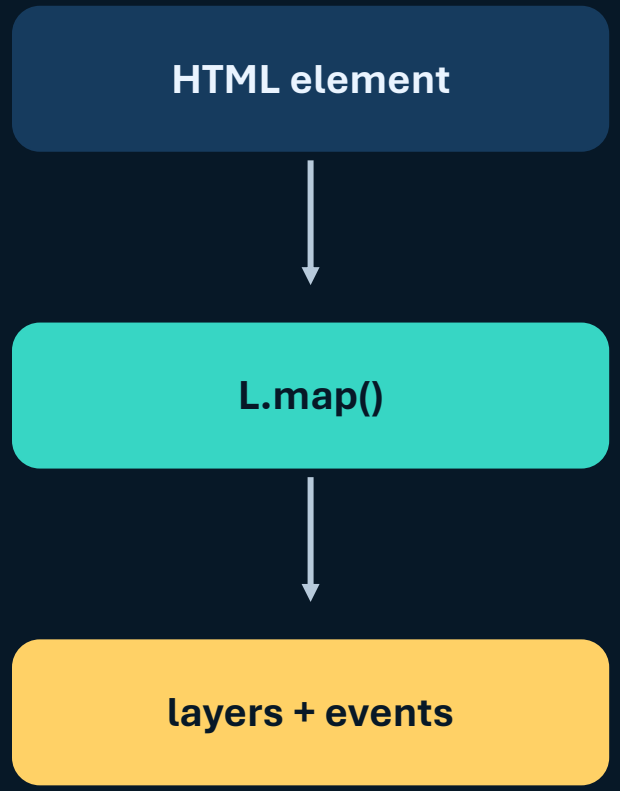


FOUNDATION

What is Leaflet?

Leaflet creates the interactive map and manages its layers, popups, and events.

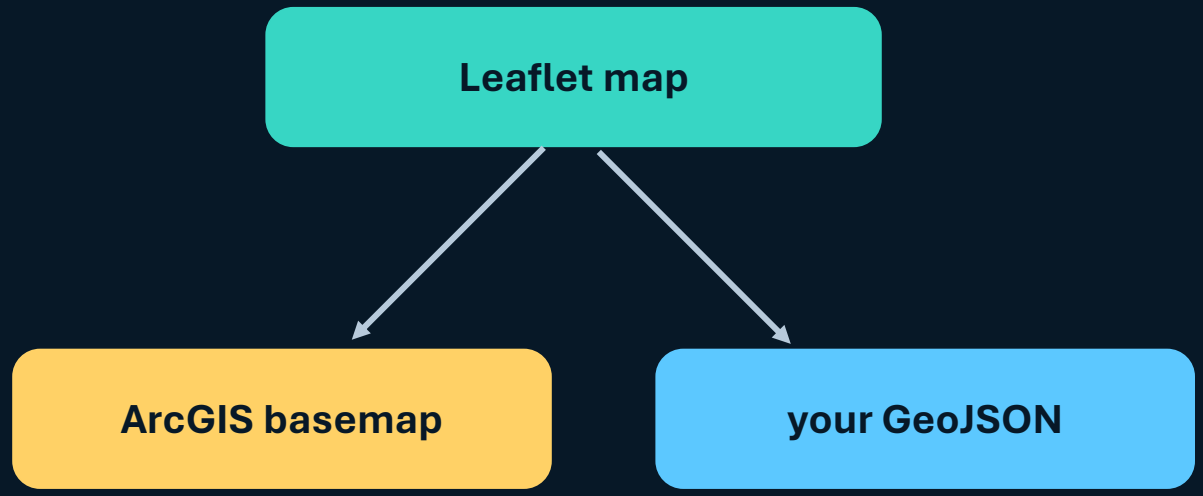
It does not automatically provide the basemap, our GeoJSON data, or the sidebar UI.



FOUNDATION

What is ArcGIS doing here?

ArcGIS supplies the basemap. Esri Leaflet and MapLibre ArcGIS connect it to our map library. Your GeoJSON stays separate.



FOUNDATION

What is GeoJSON?

GeoJSON is a feature data format. A feature combines where something is with what it means.

Geometry

Point coordinates: where

Properties

ID, name, category: what

where

what

FOUNDATIONS

Choose the geometry for the job

The real-world object and its map representation can differ.

Point

One position

A historic-place marker

LineString

A connected path

A walking route

Polygon

An enclosed area

A park boundary

FOUNDATIONS

Same location. Different array order.

Keep GeoJSON in longitude, latitude order.

Where	Array order	Mount Pocono center
Leaflet setView()	latitude, longitude	[41.1220, -75.3646]
GeoJSON	longitude, latitude	[-75.3646, 41.1220]
MapLibre center	longitude, latitude	[-75.3646, 41.1220]

FOUNDATIONS

Raster pixels. Vector features.

Tiles deliver the pieces needed for a view and zoom level.

Raster tiles

Rendered images

Colors and labels are pixels

Vector tiles

Geometry and attributes

The renderer applies a style

Local GeoJSON

An inspectable feature file

Our 21 application places

LIBRARY CHOICE

Start with requirements

These are starting points to evaluate, not exclusive rules.

Library	Consider it for	Test in your app
Leaflet	Straightforward 2D maps and web UI	Features, plugins, interactions
MapLibre GL JS	WebGL and vector styling control	Style APIs, devices, real data
OpenLayers	Varied formats and projections	Required sources and projections

We build with Leaflet and MapLibre. OpenLayers is a comparison option.

YOUR TURN • FOUNDATIONS

Read one real feature

Open Mapping Foundations and Library Choice on the workshop site.

Open data/cities/mount-pocono.geojson and inspect its first feature.

Find four things

Name

Geometry type

Longitude

Stable application ID

Our first place

Pocono Manor Historic District

Point

−75.3714875

mp-97000287

01

Build the Map

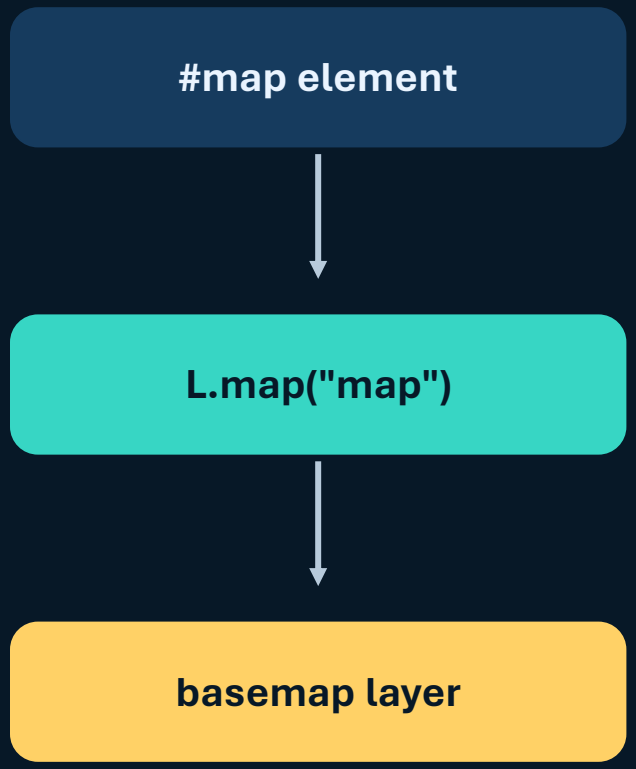
01-map.md

watch → build → checkpoint

01 · MENTAL MODEL

Container → map → layers

If the app breaks, ask which layer of the setup failed.



01 · DEBUGGING

The map can exist and still be invisible

No height means no visible map. This is the first Leaflet bug to remember.

```
<main id="map"></main>

map = L.map("map");
```

Still blank?

```
#map {
  height: 100vh;
}
```

YOUR TURN

01 · Build the Map

Open 01-map.md on the workshop site

- Load Leaflet
- Create the map container + height
- Use Mount Pocono + create the map
- Add OpenStreetMap tiles
- Swap to ArcGIS with Esri Leaflet

BUILD

18 min

Then stop at the checkpoint.

checkpoint available

02

Load Geographic Data

02-data.md

watch → build → checkpoint

02 · GEOJSON

Coordinate order will try to trick you

Leaflet setView uses latitude, longitude.
GeoJSON coordinates use longitude, latitude.

```
Leaflet: [41.1220, -75.3646]

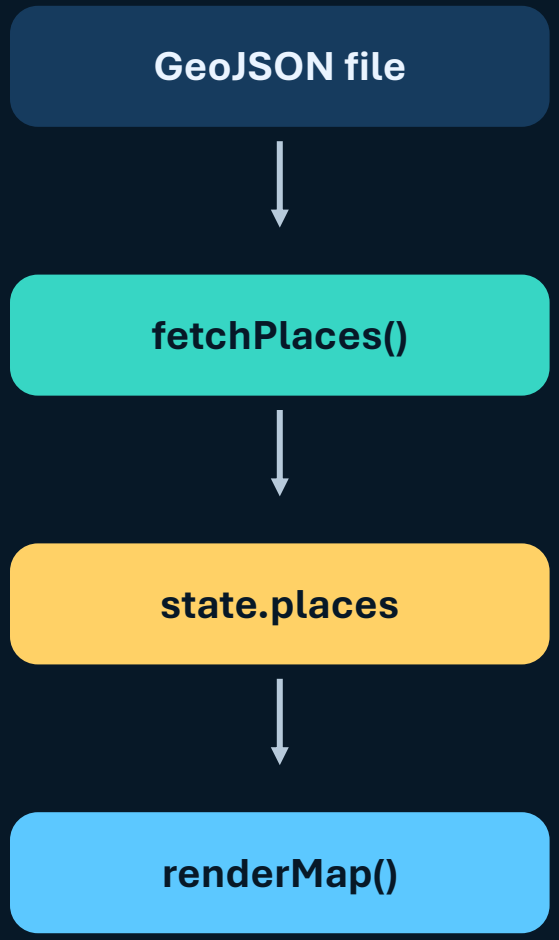
GeoJSON: [-75.3646, 41.1220]
```

same place

02 · DATA FLOW

The app owns the data

We fetch GeoJSON into state, then render it on the map. Leaflet displays the data, but it does not own our application.



YOUR TURN

02 · Load Geographic Data

Open 02-data.md on the workshop site

- Load mount-pocono.geojson
- Create state.places
- Create the GeoJSON layer
- Build popups
- Fetch and render features

BUILD

22 min

Then stop at the checkpoint.

checkpoint available

03

Build the UI

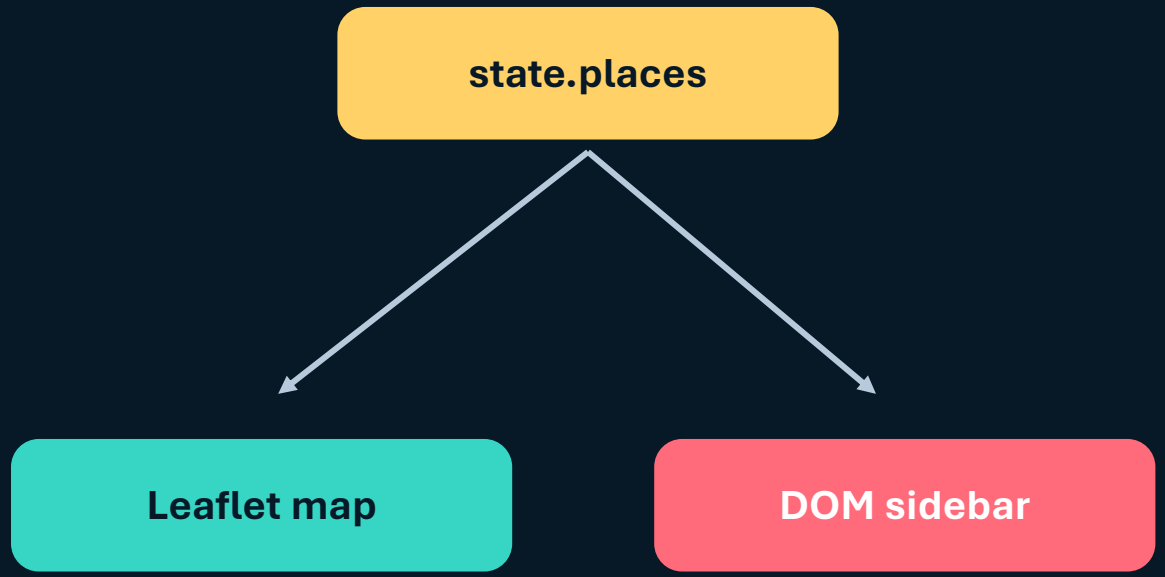
03-ui.md

watch → build → checkpoint

03 · MENTAL MODEL

The map is one view, not the whole app

Leaflet displays the data geographically. The DOM can display the same data as a normal interface.



03 · UX

Cards are future interaction points

We render cards as buttons before they do anything so the semantics are ready when Step 04 adds behavior.

```
<button class="place-card"
  data-place-id="mp-97000287">
  Pocono Manor
  Historic District
</button>
```

YOUR TURN

03 · Build the UI

Open 03-ui.md on the workshop site

- Create the app shell
- Style sidebar and cards
- Find DOM elements
- Render one card per feature
- Show the place count

BUILD

14 min

Then stop at the checkpoint.

checkpoint available

04

Synchronize Selection

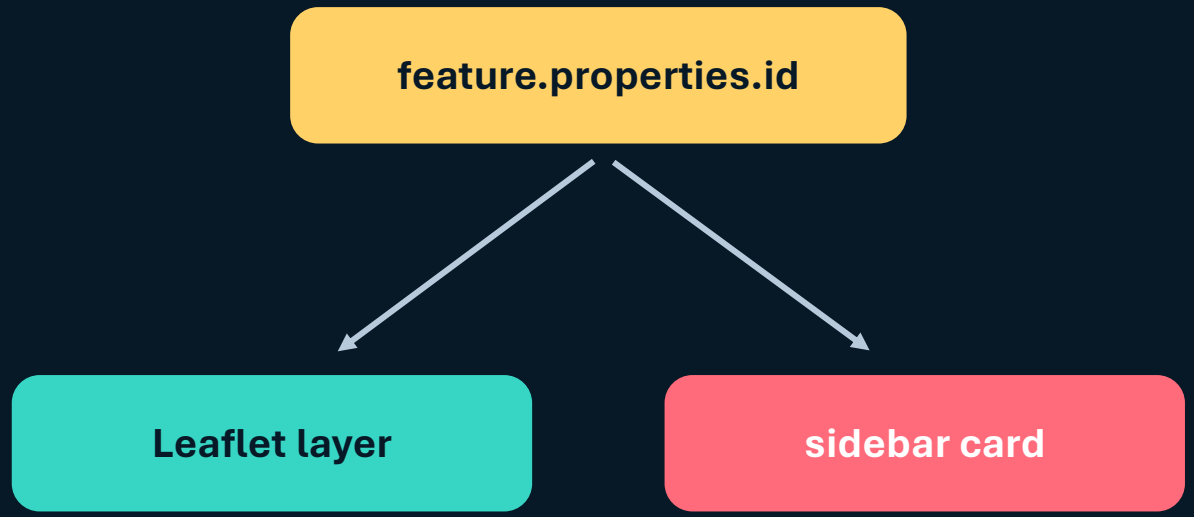
04-selection.md

watch → build → checkpoint

04 · CONNECTION

Shared IDs are the bridge

The same feature ID connects the GeoJSON feature, the Leaflet layer, and the sidebar card.

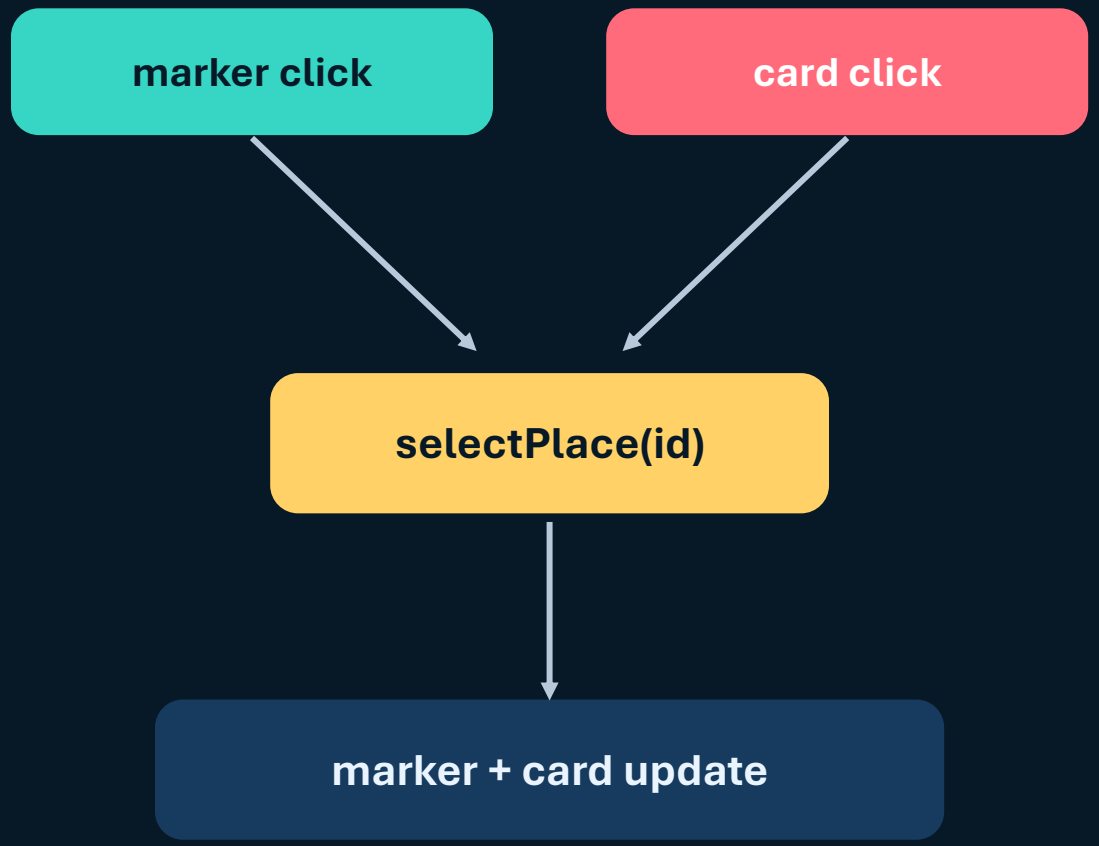



```
"mp-97000287"
```

04 · INTERACTION

One selection function, two entry points

Marker clicks and card clicks both call selectPlace. The option object lets the behavior differ slightly.



YOUR TURN

04 · Synchronize Selection

Open 04-selection.md on the workshop site

- Add selectedId
- Track layers by feature ID
- Create selectPlace()
- Style selected cards
- Listen for sidebar clicks

BUILD

21 min

Then stop at the checkpoint.

checkpoint available

REGROUP

Break

Before filtering, everyone should have selection working from both the map and the sidebar.



05

Filter Shared State

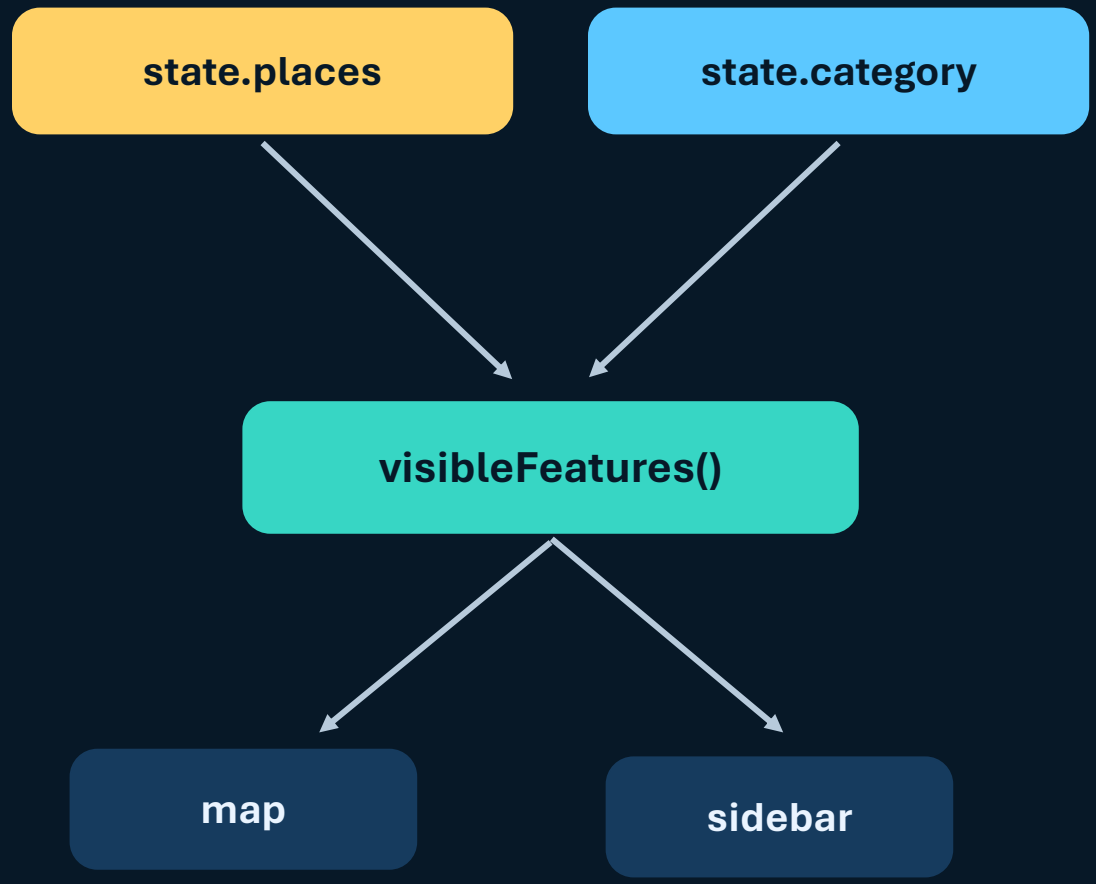
05-filtering.md

watch → build → checkpoint

05 · STATE PATTERN

Store source state. Derive visible data.

We keep state.places complete and calculate visibleFeatures from state.places plus state.category.



05 · RENDERING

One render function keeps views together

Filtering changes state. render() asks what is visible, then updates map, sidebar, and status.

```
function render() {
  const features = visibleFeatures();

  renderMap(features);
  renderList(features);
  setStatus(...);
}
```

YOUR TURN

05 · Filter Shared State

Open 05-filtering.md on the workshop site

- Add the category select
- Add category to state
- Create visibleFeatures()
- Create render()
- Clear selection when filtering

BUILD

15 min

Then stop at the checkpoint.

checkpoint available

06

Styling, Performance, and Usability

06-polish.md

watch → build → checkpoint

06 • STYLING

Make category, selection, and focus distinct

Color communicates category. Selection and focus need their own signals.

Category

Five category colors

Restore the previous color

Selection

Larger yellow marker

Dark outline + card border

Keyboard focus

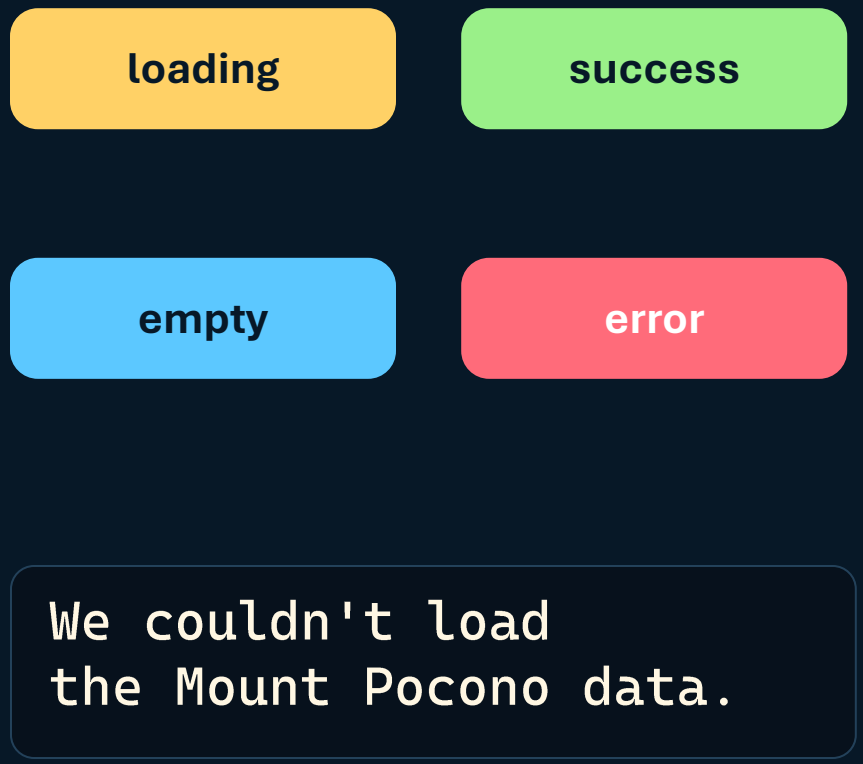
Visible control outline

Tab, Enter, and Space

06 · UX STATES

Real apps have states

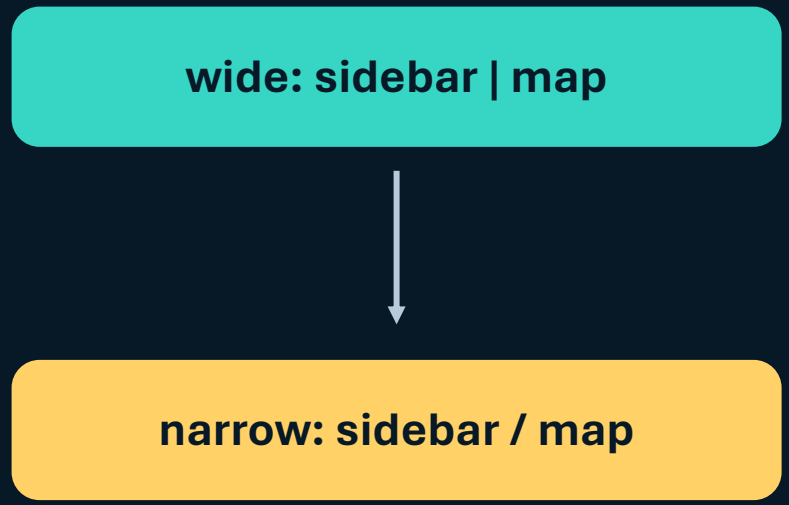
Loading, success, empty, and error are not edge cases. They are part of the user experience.



06 · RESPONSIVE

Same HTML, different layout

CSS can rearrange the sidebar and map without rebuilding the application.




```
@media (max-width: 760px)
```

06 • USABILITY

Keep the app usable as conditions change

Responsive layout, useful status, keyboard access, and reduced motion.

Controls

- Real buttons
- Visible focus
- Enabled after loading

Layout and status

- Sidebar above map on mobile
- Map retains height
- Polite status updates

Reduced motion

- Immediate map change
- Immediate card scrolling
- Respect user preference

06 • PERFORMANCE

Measure the work before changing it

Twenty-one places illustrate the method, not a scalability benchmark.

Inspect requests

GeoJSON loads once

Filter in application memory

Tile requests are separate

Compare rendering

Try preferCanvas for paths

Time synchronous render() work

Check behavior and real workloads

YOUR TURN

06 · Polish the Application

Open 06-polish.md on the workshop site

- Style categories, selection, and focus
- Test loading, empty, and error states
- Support narrow screens and reduced motion
- Compare SVG and Canvas paths
- Check requests and synchronous rendering

BUILD

24 min

Then stop at the checkpoint.

checkpoint available

Done

Leaflet App Complete

After 06-polish.md

Keep this app. Next: MapLibre.

07

MapLibre Lab

07-maplibre.md

watch → build → checkpoint

07 • MENTAL MODEL

A source holds data. A layer draws it.

One source can support different visual representations.

GeoJSON source

type: "geojson"
Full FeatureCollection
21 historic places

Circle style layer

References the source ID
Radius, fill, and stroke
Category match expression

Map interaction

Click the circle layer
Read its feature
Open a popup

07 • LEAFLET AND MAPLIBRE

Translate the jobs, not every line

The application concepts survive a change of map API.

Job	Leaflet	MapLibre
Create map	L.map()	new maplibregl.Map()
Supply GeoJSON	L.geoJSON()	addSource()
Draw and style points	Circle markers + options	Circle layer + expressions
Change visible category	Render a derived array	setFilter() on the layer
Show place details	Bind popup to layer	Popup from layer-click event

07 • STARTUP

Load the style before adding your layer

The lab imports libraries from pinned CDN modules, without a build step.

Register load, then apply style

```
map.once("load", loadPlaces);
BasemapStyle.applyStyle(map, {
  style: "arcgis/light-gray",
  token: apiKey
});
```

Then load application places

After the load event:

- fetch the GeoJSON, then add
- the source and circle layer.

07 • FILTERING

Filter the layer. Preserve the source.

Use the same state.category to derive the visible count.

Application choice

state.category

"Residential"

Map layer

map.setFilter(...)

Draw matching features

Status count

visibleFeatures().length

YOUR TURN

07 · MapLibre Lab

Open 07-maplibre.md on the workshop site

- Create maplibre-work
- Copy config.js and GeoJSON
- Create the map + apply its basemap
- Add a source, circle layer, and popups
- Test category filtering + count

BUILD

26 min

Then stop at the checkpoint.

checkpoint available

WRAP-UP • APPLY THE CHOICE

Which library would you evaluate first?

Choose a project and name two requirements you would test.

Neighborhood guide

Places + popups + web UI

Evaluate Leaflet first

Styled transport map

Vector styling + animation

Evaluate MapLibre first

Mixed-format map

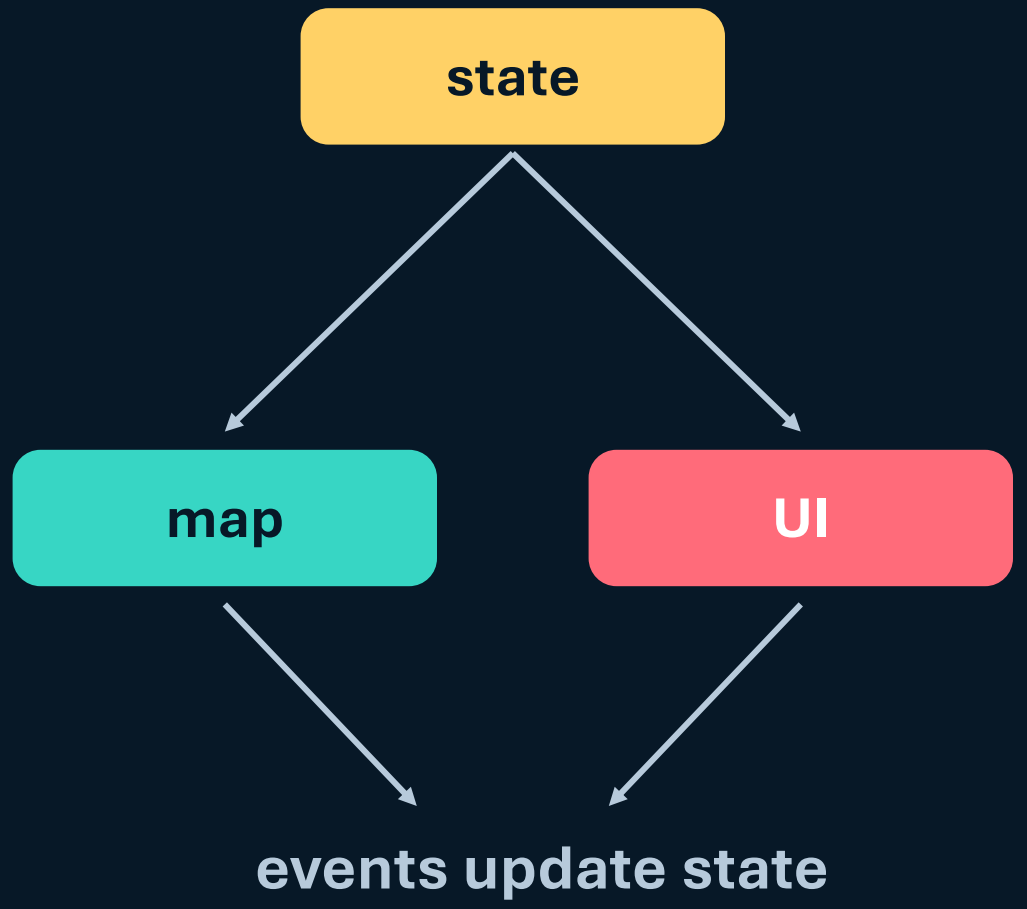
Multiple formats/projections

Evaluate OpenLayers first

FINAL MENTAL MODEL

State drives the experience

The map is powerful, but it is still one view of the application.



MAKE IT YOURS

Extensions are where the app becomes yours

Optional modules and favorites come next. The service extensions use the modular Leaflet app.

- ArcGIS geocoding
- ArcGIS Feature Layers
- Draw and export
- Your own GeoJSON
- Lines and polygons

REFERENCES

Keep these references close

The workshop site is your lab manual; the official docs explain the APIs.

- Workshop instructions
- Leaflet API reference
- MapLibre Map API
- MapLibre ArcGIS plugin
- Courtney on GitHub